



Actividad 3. Proyecto.

Perfeccionamiento del sistema de control del ascensor industrial ACME mediante instrumentación avanzada

AUTOR / INTEGRANTES:

Jorge Añón Fayos.

Cristina Almendro Barquero

Paul Abimelec Alvarez Salas

Daniel Amores Gómez

Guillermo Gonzalo Aldaz Amaguaña

INDICE

- 1. INTRODUCCIÓN Y OBJETIVOS2
 - 1.1. ENLACE A GTHUB Y WOKWI2
- 1. SELECCIÓN Y ADAPTACIÓN DEL SISTEMA DE INSTRUMENTACIÓN AVANZADA3
 - 1.2. Memoria EEPROM3
 - 1.3. Lógica difusa3
- 2. DISEÑO DEL CIRCUITO5
 - 2.1. Descripción de la infraestructura5
 - 2.2. Justificación de la distribución de pines.....6
- 3. EFICIENCIA DEL CÓDIGO.....6
- 4. DETECCIÓN DE ERRORES Y MODIFICACIONES DE DISEÑO8

1. INTRODUCCIÓN Y OBJETIVOS

El objetivo principal de esta actividad es perfeccionar el sistema de control climático y mecánico del ascensor industrial simulado en Wokwi, implementando estrategias avanzadas de instrumentación programable que optimicen su eficiencia y adaptabilidad.

Para alcanzar este objetivo general, se establecen las siguientes metas específicas:

- Implementar un sistema de supervisión inteligente integrando un algoritmo de Lógica Difusa mediante la librería eFLL para la toma de decisiones en el sistema de ventilación.
Esto sustituirá el control clásico ON/OFF por un ajuste dinámico y proporcional (PWM) de la velocidad del ventilador según la diferencia térmica que se solicite.
- Establecer control remoto inalámbrico, para ello se integra un receptor de Infrarrojos (IR) que permite al operador modificar la temperatura deseada y seleccionar las plantas de destino del ascensor a distancia mediante un mando inalámbrico.
- Garantizar la persistencia y seguridad de datos, para este fin se utiliza una memoria no volátil EEPROM del microcontrolador para almacenar la temperatura deseada por el usuario, asegurando que el sistema sea inmune a cortes en la tensión de alimentación eléctrica.
- Optimizar el rendimiento del firmware desarrollando un código completamente eficiente y no bloqueante mediante el uso de temporizadores por hardware.

1.1. ENLACE A GTHUB Y WOKWI

Enlace de Github:

https://github.com/amoresdaniel/Actividad3_Instrumentacion_electronica.git

Enlace de Wokwi: <https://wokwi.com/projects/464945624295109633>

1. SELECCIÓN Y ADAPTACIÓN DEL SISTEMA DE INSTRUMENTACIÓN AVANZADA

En esta fase del proyecto se ha perfeccionado el sistema de funcionamiento del ascensor integrando tres tecnologías de instrumentación avanzada: un algoritmo de lógica difusa para el control proporcional de la velocidad del ventilador, un mando de control remoto por infrarrojos para la interacción externa inalámbrica, y la gestión de la memoria no volátil EEPROM para asegurar la persistencia de las configuraciones críticas del usuario.

A continuación, se detalla la implementación de este sistema avanzado a través de la descripción de sus bloques de código fundamentales:

1.2. Memoria EEPROM

Para evitar que los parámetros de confort térmico configurados se borren ante un fallo eléctrico, se implementa el uso de la memoria no volátil EEPROM. Al arrancar el microcontrolador, el sistema recupera automáticamente la última temperatura deseada, puesto que la guardó. Además, en caso de detectar una temperatura fuera de un rango considerado viable o una temperatura no numérica establece por defecto la temperatura a 25 grados.

```
// Lectura de EEPROM
EEPROM.get(Memoria, TempDeseada);
Serial.print("--- DIAGNOSTICO EEPROM: Temp deseada = "); Serial.println(TempDeseada);

if (isnan(TempDeseada) || TempDeseada < 10.0 || TempDeseada > 40.0) {
    TempDeseada = 25.0;
    EEPROM.put(Memoria, TempDeseada);
}
```

1.3. Lógica difusa

La lógica difusa consiste en traducir variables físicas reales a conceptos programables mediante funciones de pertenencia trapezoidales. En este bloque se construye el universo de discurso para el "Error de Temperatura" (Temperatura Actual - Temperatura Deseada).

El concepto consiste en hacer que cuanto mayor sea la diferencia entre la temperatura actual menos la temperatura deseada el ventilador ha de girar mas deprisa. Para ello se generan tres trapecios y cada uno de ellos define unos puntos y determina el nivel de variación si es poco, normal o mucho y en base a ello se determina si el ventilador ha de girar lento, normal o rápido.

Los trapecios se configuran en este punto:

```

FuzzyInput *errorTemp = new FuzzyInput(1);

FuzzySet *errorPoco = new FuzzySet(1.5, 2.0, 4.0, 6.0); //Trapecio 1.
errorTemp->addFuzzySet(errorPoco);
FuzzySet *errorMedio = new FuzzySet(4.0, 6.0, 8.0, 10.0); //Trapecio 2.
errorTemp->addFuzzySet(errorMedio);
FuzzySet *errorMucho = new FuzzySet(8.0, 10.0, 100.0, 100.0); //Trapecio 3. Cubre hasta 100°C de error
errorTemp->addFuzzySet(errorMucho);

fuzzy->addFuzzyInput(errorTemp); //Guarda los trapecios generados

```

Después se configuran las velocidades del ventilador:

```

FuzzyOutput *velocidadVentilador = new FuzzyOutput(1);

FuzzySet *velLenta = new FuzzySet(0, 0, 80, 120);
velocidadVentilador->addFuzzySet(velLenta);
FuzzySet *velMedia = new FuzzySet(80, 120, 180, 220);
velocidadVentilador->addFuzzySet(velMedia);
FuzzySet *velRapida = new FuzzySet(180, 220, 255, 255);
velocidadVentilador->addFuzzySet(velRapida);

fuzzy->addFuzzyOutput(velocidadVentilador); //Carga las salidas generadas

```

Por último, se configuran las reglas de correlación entre las entradas y las salidas generadas:

```

//Creación de las reglas para configurar la relación entre las entradas guardadas y las salidas
FuzzyRuleAntecedent *ifErrorPoco = new FuzzyRuleAntecedent();
ifErrorPoco->joinSingle(errorPoco);
FuzzyRuleConsequent *thenVelLenta = new FuzzyRuleConsequent();
thenVelLenta->addOutput(velLenta);
FuzzyRule *fuzzyRule1 = new FuzzyRule(1, ifErrorPoco, thenVelLenta);
fuzzy->addFuzzyRule(fuzzyRule1);

FuzzyRuleAntecedent *ifErrorMedio = new FuzzyRuleAntecedent();
ifErrorMedio->joinSingle(errorMedio);
FuzzyRuleConsequent *thenVelMedia = new FuzzyRuleConsequent();
thenVelMedia->addOutput(velMedia);
FuzzyRule *fuzzyRule2 = new FuzzyRule(2, ifErrorMedio, thenVelMedia);
fuzzy->addFuzzyRule(fuzzyRule2);

FuzzyRuleAntecedent *ifErrorMucho = new FuzzyRuleAntecedent();
ifErrorMucho->joinSingle(errorMucho);
FuzzyRuleConsequent *thenVelRapida = new FuzzyRuleConsequent();
thenVelRapida->addOutput(velRapida);
FuzzyRule *fuzzyRule3 = new FuzzyRule(3, ifErrorMucho, thenVelRapida);
fuzzy->addFuzzyRule(fuzzyRule3);

```

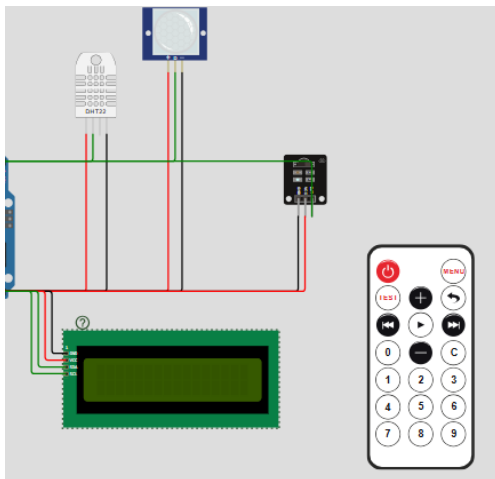
2. DISEÑO DEL CIRCUITO

El diseño expande el circuito de las actividades anteriores mediante la integración de un servomotor, garantizando que el hardware responda al firmware.

A continuación, se detalla la topología física del circuito, la distribución del conexionado eléctrico y la justificación de los pines empleados.

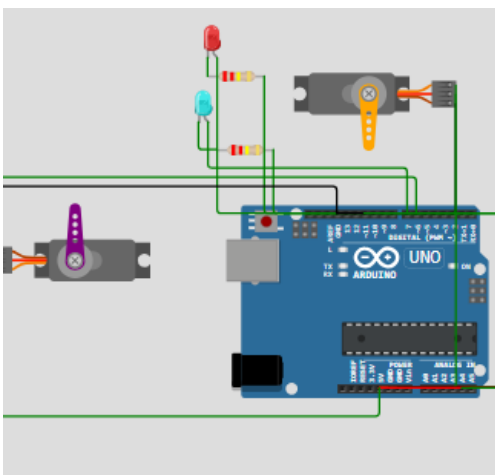
2.1. Descripción de la infraestructura

El núcleo de procesamiento está constituido por una tarjeta de desarrollo Arduino Uno. A este nodo central se conectan los subsistemas de adquisición de datos (sensores), interacción con el operador (interfaces de entrada/salida) y los actuadores encargados de simular la modificación del entorno físico:



Subsistema de entrada de señales y control remoto compuesto por un sensor digital de temperatura y humedad DHT22, un sensor volumétrico de presencia PIR encargado de la detección de personal dentro de la cabina mediante radiación infrarroja pasiva, y un receptor de Infrarrojos (IR) acoplado al pin 11 para la decodificación de las señales inalámbricas del mando a distancia industrial.

Subsistema HMI local formado por una pantalla LCD. La pantalla se acopla mediante un módulo controlador I2C (PCF8574T), lo que reduce el cableado tradicional de 6 hilos a una configuración simple de un bus serie bidireccional de dos hilos.



Subsistema de simulación de ventilador y ascensor, integra dos servomotores independientes. El primero (conectado al Pin 9) modela mecánicamente el posicionamiento del ascensor entre las 5 plantas mediante ángulos de giro definidos en su matriz física de estado. El segundo servomotor (conectado al Pin 6) se introduce en esta fase como el ventilador. La señalización lumínica se compone de un LED Rojo para el modo calefacción y un LED Azul para monitorizar el estado encendido/apagado (ON/OFF) del ventilador.

2.2. Justificación de la distribución de pines

Interrupciones externas por hardware (Pin 2): El sensor PIR se ha conectado obligatoriamente al Pin 2 de Arduino Uno. Este pin está asociado físicamente a la interrupción por hardware INT0.

PWM (Pines 6 y 9): Los servomotores requieren pulsos periódicos de ancho variable de alta precisión para mantener sus posiciones angulares estables de manera autónoma.

Ahorro de Líneas de E/S mediante bus I2C (Pines A4/A5): El microcontrolador ATmega328P dispone de pines analógicos que comparten funciones lógicas secundarias. Al mapear la interfaz de pantalla LCD a través del bus de hardware Two-Wire Interface (TWI) localizado en los pines analógicos A4 (SDA) y A5 (SCL), se reduce drásticamente el consumo de pines digitales, dejando espacio libre para la expansión remota del receptor IR (Pin 11) y los LEDs de control de potencia.

3. EFICIENCIA DEL CÓDIGO

Para lograr un sistema verdaderamente multitarea e integrado, se implementa una máquina de estados controlada de manera asíncrona mediante la función nativa `millis()`. Esto permite que el lazo principal (`void loop()`) se ejecute miles de veces por segundo, gestionando de forma simultánea e independiente tres tiempos de ejecución críticos.

- Lazo inmediato escucha constante y decodifica en tiempo real el receptor infrarrojo para capturar los comandos del mando a distancia. Esta función se ejecuta con `void loop()`

```

void loop() {
  if (IrReceiver.decode()) {
    int comando = IrReceiver.decodedIRData.command;
    switch (comando) {
      case 48: PlantaDestino = 1; break;
      case 24: PlantaDestino = 2; break;
      case 122: PlantaDestino = 3; break;
      case 16: PlantaDestino = 4; break;
      case 56: PlantaDestino = 5; break;
      case 2: // Subir Tempreatura deseada
        TempDeseada += 1.0;
        EEPROM.put(Memoria, TempDeseada);
        Serial.print("Nueva Tempreatura deseada: "); Serial.println(TempDeseada);
        break;
      case 152: // Bajar Tempreatura deseada
        TempDeseada -= 1.0;
        EEPROM.put(Memoria, TempDeseada);
        Serial.print("Nueva Tempreatura deseada: "); Serial.println(TempDeseada);
        break;
    }
    IrReceiver.resume();
  }
}

```

- Lazo Dinámico continuo movimiento grado a grado del servomotor del ventilador, cuya velocidad se recalcula dinámicamente según las órdenes del motor difuso.

```

if (velocidadVentiladorActual > 0) {
  int retardoPaso = map(velocidadVentiladorActual, 1, 255, 40, 2);
  if (millis() - tiempoAnteriorServo >= retardoPaso) {
    tiempoAnteriorServo = millis();
    anguloVentilador += direccionVentilador;

    if (anguloVentilador >= 180) {
      anguloVentilador = 180;
      direccionVentilador = -1;
    } else if (anguloVentilador <= 0) {
      anguloVentilador = 0;
      direccionVentilador = 1;
    }
    ventiladorServo.write(anguloVentilador);
  }
}

```

- Lazo periódico cada 500 ms se realiza la lectura del sensor DHT22, cálculo de las ecuaciones de inferencia difusa, refresco de las variables de estado e impresión en el monitor serie de diagnóstico.

```

if (TiempoActual - TiempoAnterior >= Intervalo) {
  TiempoAnterior = TiempoActual;

  float Temperatura = dht.readTemperature();
}

```


Uso de una interrupción para capturar la presencia de personas en el ascensor.

```
//Interrupción
void isr_Presencia() {
|  PresenciaDetectada = true;
}
```

4. DETECCIÓN DE ERRORES Y MODIFICACIONES DE DISEÑO

Durante la fase de integración y simulación en el entorno Wokwi, se observó un mal funcionamiento del sistema simulado.

A) Colapso en la Lógica Difusa

Problema: En las primeras iteraciones, la simulación colapsaba por completo cuando la temperatura alcanzaba ciertos valores intermedios o extremos.

Tras analizar en detalle se identificó que las funciones que generan los trapecios de entrada no se solapaban correctamente. Esto creaba huecos donde el grado de pertenencia era 0 para todas las reglas. Cuando la librería eFLL intentaba ejecutar la defuzzificación el denominador de la ecuación era cero, provocando la parada del servomotor que simula el ventilador.

Solución: Se rediseñaron las coordenadas de los trapecios garantizando un solapamiento continuo. Además, se extendió el límite del último trapecio hasta un valor irreal (100.0 °C) para atrapar un rango mayor de lo que suele ser normal.

B) Saturación por hardware del servomotor (que simula al ventilador)

Problema: Cuando el sistema de climatización alcanzaba la temperatura óptima y el algoritmo difuso dictaba una velocidad de ventilación nula (PWM = 0), el simulador sufría problemas a la hora de actualizar de nuevo los valores.

El lazo dinámico del código estaba enviando la instrucción ventiladorServo.write(0) de manera ininterrumpida. Esta sobresaturación del canal PWM hacía que el servomotor colapsase y no actualizase correctamente su posición.

Solución: Se implementó una cláusula de bloqueo o guarda de estado (if (anguloVentilador != 0)) en el lazo principal. Esta modificación asegura que, una vez que el actuador ha regresado a su posición de reposo, el microcontrolador cese de enviar señales redundantes, liberando carga de CPU: